

ArrangerActor.scala :

```
package upmc.akka.ppc
```

```
import akka.actor.{Actor, ActorRef, Props}
```

```
import scala.concurrent.ExecutionContext.Implicits.global
```

```
/**
```

```
 * L'ArrangerActor reçoit un indice déterminé par le Maestro (en fonction des dés),
```

```
 * et utilise deux matrices pour sélectionner un indice de mesure, qu'il demande ensuite au DataBaseActor.
```

```
 */
```

```
class ArrangerActor(maestroRef: ActorRef) extends Actor {
```

```
  import DataBaseActor._
```

```
  // Compteur du nombre de demandes effectuées, sert à déterminer quelle matrice utiliser
```

```
  private var requestCounter = 0
```

```
  // Acteur de base de données, fournissant les mesures correspondantes
```

```
  private val dbActor = context.actorOf(Props[DataBaseActor], "scoreDatabase")
```

```
  // Matrices pour sélectionner les mesures en fonction de l'indice (résultat des dés)
```

```
  // Première section du morceau
```

```
  private val firstSection: Array[Array[Int]] = Array(
```

```
    Array( 96, 22, 141, 41, 105, 122, 11, 30),
```

```
    Array( 32, 6, 128, 63, 146, 46, 134, 81),
```

```
    Array( 69, 95, 158, 13, 153, 55, 110, 24),
```

```
    Array( 40, 17, 113, 85, 161, 2, 159, 100),
```

```
    Array(148, 74, 163, 45, 80, 97, 36, 107),
```

```
    Array(152, 60, 171, 53, 99, 133, 21, 127),
```

```
    Array(119, 84, 114, 50, 140, 86, 169, 94),
```

```
    Array( 98, 142, 42, 156, 75, 129, 62, 123),
```

```
    Array( 3, 87, 165, 61, 135, 47, 147, 33),
```

```
    Array( 54, 130, 10, 103, 228, 37, 106, 5)
```

```
)
```

```
// Deuxième section du morceau
```

```
private val secondSection: Array[Array[Int]] = Array(
```

```
  Array( 70, 121, 26, 9, 112, 49, 109, 14),
```

```
  Array( 11, 7, 39, 126, 56, 174, 18, 116),
```

```
  Array( 83, 66, 139, 15, 132, 73, 58, 145),
```

```
  Array( 79, 90, 176, 7, 34, 67, 160, 52),
```

```
  Array(170, 25, 143, 64, 125, 76, 136, 1),
```

```
  Array( 93, 138, 71, 150, 29, 101, 162, 23),
```

```
  Array(151, 16, 155, 57, 175, 43, 168, 86),
```

```
  Array(172, 120, 88, 48, 166, 51, 115, 72),
```

```
  Array(111, 65, 77, 19, 82, 137, 38, 149),
```

```
  Array( 8, 102, 4, 31, 164, 144, 59, 173),
```

```
  Array( 78, 35, 20, 108, 92, 12, 124, 44)
```

```
)
```

```
def receive: Receive = {
```

```
  case GetMeasure(diceValue) =>
```

```
// Détermine la partie (section) du morceau en fonction du compteur
```

```
  val partIndex = requestCounter % 16
```

```
  val measureIndex =
```

```
    if (partIndex < 8) firstSection(diceValue)(requestCounter % 8)
```

```
    else secondSection(diceValue)(requestCounter % 8)
```

```
  requestCounter += 1
```

```
// Demande la mesure correspondante au DataBaseActor
```

```
  dbActor ! GetMeasure(measureIndex - 1)
```

```
  case Measure(chordSet) =>
```

```
// Une fois la mesure obtenue, on la renvoie au Maestro
```

```
maestroRef ! Measure(chordSet)

}

}
```

StyleExport

verified_user

DataBase.scala:

```
package upmc.akka.ppc

import akka.actor._

object DataBaseActor {

  abstract class ObjetMusical

  case class Note (pitch: Int, dur: Int, vol: Int) extends ObjetMusical

  case class Chord (date: Int, notes: List[Note]) extends ObjetMusical

  case class Measure (chords: List[Chord]) extends ObjetMusical

  case class GetMeasure (num: Int)

  case class Start()

  val measures1 : List [Measure] = List (

    Measure (List (Chord (0 , List (Note(42 ,610, 86), Note(54 ,594, 81), Note(81 ,315, 96))),

    Chord (292 , List (Note(78 ,370, 78))),

    Chord (601 , List (Note(76 ,300, 91), Note(43 ,585, 83), Note(55 ,588, 98))),

    Chord (910 , List (Note(79 ,335, 96))),

    Chord (1189 , List (Note(73 ,342, 86), Note(57 ,595, 76), Note(45 ,607, 83))),

    Chord (1509 , List (Note(76 ,280, 93))))) ,

    Measure (List (Chord (0 , List (Note(49 ,616, 86))),

    Chord (295 , List (Note(64 ,314, 78))),
```

Chord (583 , List (Note(52 ,616, 88), Note(68 ,296, 81))),

Chord (863 , List (Note(69 ,290, 96))),

Chord (1168 , List (Note(57 ,607, 79), Note(73 ,305, 79))),

Chord (1473 , List (Note(76 ,310, 100)))))

Maestro.scala:

```
package upmc.akka.ppc
```

```
import akka.actor.{Actor, Props}
```

```
import scala.concurrent.duration._
```

```
import scala.concurrent.ExecutionContext.Implicits.global
```

```
object Maestro {
```

```
  case object StartPerformance
```

```
}
```

```
/**
```

```
 * Le Maestro est responsable du déroulement global de la performance.
```

```
 * Il lance la demande de mesures musicales, obtient les données depuis l'ArrangerActor,
```

```
 * puis transmet ces mesures au PlayerActor pour la reproduction sonore.
```

```
 */
```

```
class Maestro extends Actor {
```

```
  import DataBaseActor._
```

```
  import PlayerActor._
```

```
  import Maestro._
```

```
  // Création de l'acteur en charge de la performance musicale et de l'acteur en charge de l'arrangement
```

```
  private val musician = context.actorOf(Props[PlayerActor], "musician")
```

```
  private val arranger = context.actorOf(Props(new ArrangerActor(self)), "arranger")
```

```
  // Générateurs de nombres aléatoires pour décider des mesures à jouer
```

```

private val diceA = new scala.util.Random

private val diceB = new scala.util.Random

// Intervalle de temps entre chaque nouvelle mesure

private val baseTime = 1800.milliseconds

private val scheduler = context.system.scheduler

private var selectedNumber = 0

def receive: Receive = {

  case StartPerformance =>

    // On détermine un indice de mesure en combinant deux jets de dés

    selectedNumber = diceA.nextInt(5) + diceB.nextInt(5)

    arrange ! GetMeasure(selectedNumber)

  case Measure(chords) =>

    // Une fois la mesure récupérée depuis le "score", on la transmet au musicien

    musician ! Measure(chords)

    // Programme la prochaine demande de mesure après un certain délai

    scheduler.scheduleOnce(baseTime, self, StartPerformance)

}

}

```

Main.scala:

```

package upmc.akka.ppc

import akka.actor.{ActorSystem, Props}

/**
 * Point d'entrée de l'application.
 *
 * Initialise l'ActorSystem, crée le Maestro et lance la performance.
 */

object Concert extends App {

```

```
// Initialisation de l'ActorSystem

val system = ActorSystem("MozartDiceGameSystem")

// Création du Maestro (remplace l'ancien Conductor)

val maestroRef = system.actorOf(Props[Maestro], "maestro")

// Lancement de la performance

maestroRef ! Maestro.StartPerformance

}
```

Player.scala:

```
package upmc.akka.ppc

import javax.sound.midi._

import javax.sound.midi.ShortMessage._

import scala.concurrent.ExecutionContext

import scala.concurrent.duration._

import ExecutionContext.Implicits.global

import akka.actor.{Actor, Props, ActorSystem}

object PlayerActor {

  case class MidiNote(pitch: Int, vel: Int, dur: Int, at: Int)

  // Tentative de récupération du synthétiseur "Gervill"

  val synthInfo = MidiSystem.getMidiDeviceInfo().find(_.getName == "Gervill")

  val device = synthInfo.map(MidiSystem.getMidiDevice).getOrElse {

    println("[ERREUR] Synthétiseur Gervill introuvable.")

    sys.exit(1)

  }

  val receiver = device.getReceiver()
```

```
def sendNoteOn(pitch: Int, velocity: Int, channel: Int): Unit = {
```

```
  val onMsg = new ShortMessage
```

```
  onMsg.setMessage(NOTE_ON, channel, pitch, velocity)
```

```
  receiver.send(onMsg, -1)
```

```
}
```

```
def sendNoteOff(pitch: Int, channel: Int): Unit = {
```

```
  val offMsg = new ShortMessage
```

```
  offMsg.setMessage(NOTE_ON, channel, pitch, 0)
```

```
  receiver.send(offMsg, -1)
```

```
}
```

```
}
```

```
/**
```

```
 * Le PlayerActor (musicien) reçoit des mesures contenant plusieurs accords.
```

```
 * Chaque accord contient des notes, avec leur hauteur, volume, durée et date de début.
```

```
 * Le PlayerActor planifie l'envoi des évènements MIDI (NOTE_ON / NOTE_OFF) au bon moment.
```

```
 */
```

```
class PlayerActor extends Actor {
```

```
  import DataBaseActor._
```

```
  import PlayerActor._
```

```
  device.open()
```

```
  def receive: Receive = {
```

```
    case Measure(chords) =>
```

```
      println(s"[PlayerActor] Mesure reçue, contenant ${chords.size} accords. Programmation des notes...")
```

```
      // Pour chaque accord et chaque note, on programme l'émission des messages MIDI
```

```
      chords.foreach { chord =>
```

```
        chord.notes.foreach { note =>
```

```
          val midiEvt = MidiNote(note.pitch, note.vol, note.dur, chord.date)
```

```

    self ! midiEvt
  }

}

case MidiNote(p, v, d, start) =>

  // Programme l'envoi des messages NOTE_ON / NOTE_OFF aux moments appropriés

  context.system.scheduler.scheduleOnce(start.milliseconds) {

    sendNoteOn(p, v, 10)

  }

  context.system.scheduler.scheduleOnce((start + d).milliseconds) {

    sendNoteOff(p, 10)

  }

}
}

```

一、代码部分详解 / Explication des parties de code

1. ArrangerActor.scala

中文说明：

- **作用：**

ArrangerActor 作为一个 Akka Actor，负责接收由 Maestro（指挥家）发来的请求。它根据传入的骰子值（diceValue）和一个计数器 requestCounter，利用两个预定义的二维数组（矩阵）来选择乐曲中的小节编号。选择完后，它向 DataBaseActor 发送 GetMeasure 消息，请求获取相应小节的音乐数据；当收到小节数据（Measure 消息）后，再将该数据转发给 Maestro。

- **重点细节：**

- 使用了 **requestCounter** 来确定使用哪一个矩阵（前8次使用 firstSection，后8次使用 secondSection）。
- 两个矩阵分别代表乐曲的两个部分（段落）。
- 根据骰子结果（作为数组的行索引）和计数器模运算（确定列索引）选取小节编号。
- 发送消息的顺序：先发送 GetMeasure 给数据库演员，再在接收到 Measure 后将其转发给 Maestro。

French Explanation :

- **Fonction :**

L'acteur ArrangerActor reçoit des requêtes envoyées par le Maestro. En se basant sur la

valeur du dé (diceValue) et un compteur requestCounter, il utilise deux matrices pré-définies pour sélectionner l'indice d'une mesure dans la partition. Ensuite, il envoie un message GetMeasure à l'acteur DataBaseActor afin d'obtenir les données musicales correspondantes. Une fois la mesure reçue (via le message Measure), il la transmet au Maestro.

- **Points clés :**

- L'utilisation du **requestCounter** permet de déterminer quelle matrice utiliser (les 8 premiers accès utilisent firstSection, les 8 suivants secondSection).
- Les deux matrices représentent deux parties distinctes de la pièce musicale.
- Le choix de la mesure se fait en utilisant la valeur du dé (pour l'index de ligne) et une opération modulo sur le compteur (pour l'index de colonne).
- La séquence des messages est la suivante : d'abord, envoi d'un GetMeasure à l'acteur de base de données, puis, après réception d'un Measure, transmission de celui-ci au Maestro.

2. DataBase.scala

中文说明：

- **作用：**

DataBaseActor 定义了音乐数据模型和数据库演员所使用的消息。

- 定义了抽象类 ObjetMusical 和其子类：Note（音符）、Chord（和弦）以及 Measure（小节）。
- 定义了消息类型，如 GetMeasure 用于请求小节数据，以及 Start 作为开始的信号。
- 同时提供了一个 measures1 列表，包含了实际的音乐数据（两个小节示例）。

- **重点细节：**

- 数据结构的设计体现了音乐的层次：小节中包含多个和弦，每个和弦由一组音符构成。
- GetMeasure 的参数是数字索引（可能用于从列表中选择具体小节）。

French Explanation :

- **Fonction :**

Le fichier DataBase.scala définit le modèle de données musicales ainsi que les messages utilisés par l'acteur de base de données.

- Il déclare une classe abstraite ObjetMusical et ses sous-classes : Note (note musicale), Chord (accord) et Measure (mesure).
- Les types de messages tels que GetMeasure (pour demander une mesure) et Start (signal de démarrage) y sont également définis.
- Une liste measures1 est fournie, contenant des exemples concrets de données musicales (deux mesures).

- **Points clés :**

- La structure des données reflète la hiérarchie musicale : une mesure contient plusieurs accords et chaque accord est composé d'une liste de notes.
- Le message GetMeasure utilise un indice numérique pour sélectionner la mesure correspondante dans la liste.

3. Maestro.scala

中文说明：

- **作用：**

Maestro 是整个应用的指挥家，负责控制音乐演奏的流程。

- 它创建了两个子演员：PlayerActor（演奏者）和 ArrangerActor（编曲者）。
- 使用两个随机数生成器（骰子）来决定下一个演奏的小节编号，并向 ArrangerActor 发送 GetMeasure 消息。
- 当接收到从 ArrangerActor 转发过来的 Measure 消息后，将该消息发送给 PlayerActor 播放，并利用调度器（scheduler）在一定延时后触发下一次小节请求。

- **重点细节：**

- 使用 scheduler.scheduleOnce 实现定时触发，保证音乐演奏的节奏。
- 随机数生成器决定每次小节选择的随机性，模拟了“骰子游戏”的特点。

French Explanation :

- **Fonction :**

L'acteur Maestro est le chef d'orchestre responsable du déroulement global de la performance musicale.

- Il crée deux acteurs enfants : PlayerActor (le musicien) et ArrangerActor (l'arrangeur).
- Il utilise deux générateurs aléatoires (simulant des dés) pour déterminer l'indice de la mesure suivante, puis envoie un message GetMeasure à l'acteur arrangeur.
- Après avoir reçu le message Measure (transmis par l'arrangeur), il le transmet au musicien (PlayerActor) et planifie la demande de la mesure suivante avec scheduler.scheduleOnce pour respecter le rythme de la performance.

- **Points clés :**

- L'utilisation du scheduler permet de programmer l'exécution périodique des mesures.
- La logique aléatoire (décision par dés) reflète l'esprit du jeu de Mozart par dés.

4. Main.scala

中文说明：

- **作用：**

Main.scala（在本例中对象名为 Concert）是应用程序的入口点。

- 它创建了一个 Akka ActorSystem，并在其中创建了 Maestro 演员。
- 启动后，向 Maestro 发送 StartPerformance 消息以开始演奏流程。

- **重点细节：**

- ActorSystem 的初始化和演员的创建都是 Akka 应用的标准步骤。
- 应用启动后由 Maestro 统筹整个演奏流程。

French Explanation :

- **Fonction :**

Le fichier Main.scala, ici représenté par l'objet Concert, constitue le point d'entrée de l'application.

- Il initialise un ActorSystem et crée l'acteur Maestro au sein de ce système.
- Dès le démarrage, il envoie le message StartPerformance au Maestro pour lancer la performance musicale.

- **Points clés :**

- L'initialisation de l'ActorSystem et la création des acteurs sont des étapes standards dans une application Akka.
- Le Maestro orchestre l'ensemble du déroulement de la performance dès le lancement.

5. Player.scala

中文说明:

- **作用:**

PlayerActor 是实际负责播放音乐的演员，它接收包含和弦的 Measure 消息，并根据每个和弦中音符的信息发送 MIDI 消息。

- 在 PlayerActor 的伴生对象中，定义了 MidiNote 消息类型，并尝试获取 Java MIDI 系统中的 "Gervill" 合成器。
- 提供了两个工具方法 sendNoteOn 和 sendNoteOff，分别用于发送开启音符和关闭音符的 MIDI 指令。
- 在演员的 receive 方法中，当收到 Measure 消息后，会遍历和弦和其中的音符，为每个音符通过调度器 (scheduler.scheduleOnce) 安排在正确的时间发送 MIDI 消息 (NOTE_ON 与 NOTE_OFF)。

- **重点细节:**

- MIDI 的实现依赖于 Java 的 MIDI 系统，通过 MidiSystem.getMidiDeviceInfo 获取合成器信息。
- 每个音符的播放和停止都基于其“开始时间”和“持续时间”进行调度。
- 调度器使得各个 MIDI 消息能在正确的时刻被发送，从而实现实时演奏。

French Explanation :

- **Fonction :**

L'acteur PlayerActor est chargé de la reproduction sonore. Il reçoit des messages Measure contenant plusieurs accords, et pour chaque accord, il planifie l'envoi de messages MIDI basés sur les notes qui le composent.

- Dans l'objet compagnon PlayerActor, on définit la classe MidiNote ainsi que la tentative d'obtention du synthétiseur "Gervill" via le système MIDI de Java.
- Deux méthodes utilitaires, sendNoteOn et sendNoteOff, sont fournies pour envoyer les messages MIDI correspondants (activation et désactivation d'une note).
- Dans la méthode receive, dès qu'un message Measure est reçu, l'acteur parcourt chaque accord et chaque note, et utilise le scheduler (scheduler.scheduleOnce) pour envoyer les messages MIDI aux instants définis (début et fin de la note).

- **Points clés :**

- L'intégration avec le système MIDI Java permet de produire du son en temps réel.
- Le scheduling des messages MIDI garantit que les notes sont jouées et arrêtées selon leur timing défini (basé sur start et dur).
- La gestion des messages MIDI (NOTE_ON / NOTE_OFF) est cruciale pour une interprétation musicale correcte.

二、可能的考察问题及答案 / Questions d'examen potentielles et

leurs réponses

问题 1 / Question 1

中文：

Q1: ArrangerActor 的主要作用是什么？它如何利用骰子值和矩阵来选择乐曲小节？

A1:

- **回答：** ArrangerActor 负责根据 Maestro 给出的骰子值，结合内部计数器 requestCounter 来确定使用哪个矩阵以及矩阵中的哪个元素，从而选出一个小节编号。具体来说，如果 $\text{requestCounter} \% 16$ 小于 8 则使用 firstSection 矩阵，否则使用 secondSection；同时用 $\text{requestCounter} \% 8$ 选取矩阵内的列，行则由骰子值确定。然后它向 DataBaseActor 请求该编号对应的小节数据。

French :

Q1 : Quel est le rôle principal de l'acteur ArrangerActor et comment utilise-t-il la valeur du dé ainsi que les matrices pour sélectionner une mesure ?

R1 :

- **Réponse :** L'acteur ArrangerActor reçoit la valeur du dé envoyée par le Maestro et, en combinant cette valeur avec le compteur interne requestCounter, il détermine quelle matrice utiliser et quelle case de la matrice choisir pour obtenir l'indice d'une mesure. Concrètement, si $\text{requestCounter} \% 16$ est inférieur à 8, il utilise la matrice firstSection ; sinon, il utilise secondSection. L'index de colonne est déterminé par $\text{requestCounter} \% 8$ et l'index de ligne par la valeur du dé. Il envoie ensuite un message GetMeasure à DataBaseActor pour récupérer la mesure correspondante.

问题 2 / Question 2

中文：

Q2: 在 Maestro 演员中，如何控制整个音乐演奏流程？

A2:

- **回答：** Maestro 通过生成两个随机数（模拟骰子）来决定下一个要演奏的小节编号，并发送 GetMeasure 消息给 ArrangerActor。在接收到 Measure 消息后，它将该消息转发给 PlayerActor，并利用 scheduler.scheduleOnce 在设定的时间间隔后再次触发 StartPerformance 消息，从而实现循环播放。

French :

Q2 : Comment l'acteur Maestro contrôle-t-il le déroulement global de la performance musicale ?

R2 :

- **Réponse :** Le Maestro utilise deux générateurs aléatoires (simulant des lancers de dés) pour déterminer l'indice de la prochaine mesure à jouer, puis envoie un message GetMeasure à l'acteur ArrangerActor. Après avoir reçu le message Measure, il le transmet au PlayerActor et programme l'envoi d'un nouveau message StartPerformance après un délai défini à l'aide de scheduler.scheduleOnce, assurant ainsi la continuité de la performance.

问题 3 / Question 3

中文：

Q3: PlayerActor 如何安排 MIDI 消息的发送？

A3:

- **回答：** 当 PlayerActor 收到包含多个和弦的 Measure 消息时，它会遍历每个和弦和其中的每个音符，并为每个音符生成 MidiNote 消息。利用调度器 (scheduler.scheduleOnce)，在音符的开始时间发送 sendNoteOn 消息（激活音符），并在开始时间加上音符持续时间后发送 sendNoteOff 消息（关闭音符），从而确保 MIDI 消息在正确的时间点触发。

French :

Q3 : Comment l'acteur PlayerActor programme-t-il l'envoi des messages MIDI ?

R3 :

- **Réponse :** Lorsqu'il reçoit un message Measure contenant plusieurs accords, le PlayerActor parcourt chaque accord et chaque note pour générer des messages MidiNote. Ensuite, il utilise scheduler.scheduleOnce pour programmer l'envoi du message sendNoteOn à l'instant de début de la note, puis le message sendNoteOff après la durée de la note, garantissant ainsi que les messages MIDI sont envoyés au bon moment.

问题 4 / Question 4

中文：

Q4: 数据库部分 (DataBase.scala) 如何构造音乐数据？

A4:

- **回答：** 数据库部分定义了音乐数据的三个层次：Note（单个音符）、Chord（和弦，由多个音符组成）以及 Measure（小节，由多个和弦构成）。同时，它定义了消息 GetMeasure 用于请求特定小节的数据，并通过列表 measures1 存储了实际的音乐示例。

French :

Q4 : Comment la partie base de données (DataBase.scala) structure-t-elle les données musicales ?

R4 :

- **Réponse :** La partie base de données définit trois niveaux de structure musicale : Note (note individuelle), Chord (accord, composé de plusieurs notes) et Measure (mesure, composée de plusieurs accords). Elle définit également le message GetMeasure pour demander une mesure spécifique, et la liste measures1 contient des exemples concrets de données musicales.

问题 5 / Question 5

中文：

Q5: 在 Main.scala 中，如何初始化 ActorSystem 并启动整个应用？

A5:

- **回答：** Main.scala 中通过创建 ActorSystem（命名为 "MozartDiceGameSystem"）来初始化 Akka 系统，然后创建 Maestro 演员，并通过向其发送 StartPerformance 消息来启动整个演奏流程。

French :

Q5 : Comment le fichier Main.scala initialise-t-il l'ActorSystem et lance-t-il l'application ?

R5 :

- **Réponse :** Dans Main.scala, l'ActorSystem est initialisé (nommé "MozartDiceGameSystem"), puis l'acteur Maestro est créé. Enfin, en envoyant le message StartPerformance au Maestro, la performance musicale est lancée.

1. ArrangerActor.scala

问题 1

中文：

Q1: 在 ArrangerActor 中，requestCounter 如何影响 partIndex 和 measureIndex 的计算？

A1:

- **回答：**
requestCounter 用于计算 partIndex（即 $\text{requestCounter} \% 16$ ），其值决定使用哪一部分的矩阵：若小于 8 则使用 firstSection，否则使用 secondSection。随后，利用 $\text{requestCounter} \% 8$ 得到矩阵的列索引，而骰子值作为行索引，从而选出具体的 measureIndex。

Français :

Q1 : Comment le requestCounter dans ArrangerActor influence-t-il le calcul du partIndex et du measureIndex ?

R1 :

- **Réponse :**
Le requestCounter est utilisé pour calculer le partIndex via $\text{requestCounter} \% 16$. Si le résultat est inférieur à 8, la matrice firstSection est utilisée, sinon c'est secondSection. Ensuite, $\text{requestCounter} \% 8$ fournit l'indice de colonne, tandis que la valeur du dé détermine l'indice de ligne, permettant ainsi de sélectionner le measureIndex précis.

问题 2

中文：

Q2: 如果骰子传入的值不在矩阵行的有效范围内，会出现什么情况？

A2:

- **回答：**
代码中骰子值直接作为矩阵的行索引，假定其值在矩阵行数范围内（例如 0 到 9 或 0 到相应的最大行号）。如果骰子值超出范围，将导致数组下标越界错误，从而引发运行时异常。

Français :

Q2 : Que se passe-t-il si la valeur du dé utilisée comme index de ligne n'est pas dans la plage valide de la matrice ?

R2 :

- **Réponse :**

La valeur du dé est utilisée directement comme index de ligne, en supposant qu'elle est comprise dans la plage valide (par exemple de 0 à 9 ou autre limite définie). Si la valeur dépasse cette plage, cela provoquera une exception d'index hors limites lors de l'accès à l'élément du tableau.

问题 3

中文:

Q3: 描述 ArrangerActor 如何与 DataBaseActor 和 Maestro 之间传递消息完成数据流。

A3:

- **回答:**

ArrangerActor 首先接收到 Maestro 发来的 GetMeasure(diceValue) 消息，经过计算后向内部创建的 DataBaseActor 发送 GetMeasure(measureIndex - 1) 请求。当 DataBaseActor 返回 Measure(chordSet) 消息后，ArrangerActor 将其转发给 maestroRef (Maestro)，完成数据传递。

Français :

Q3 : Expliquez comment l'acteur ArrangerActor transmet les messages entre DataBaseActor et Maestro pour acheminer les données.

R3 :

- **Réponse :**

ArrangerActor reçoit d'abord un message GetMeasure(diceValue) du Maestro, calcule l'indice de la mesure en se servant de ses matrices, puis envoie un message GetMeasure(measureIndex - 1) à l'acteur DataBaseActor. Une fois que DataBaseActor renvoie un message Measure(chordSet), ArrangerActor transmet ce message au Maestro (via maestroRef).

2. DataBase.scala

问题 1

中文:

Q1: 如何在 DataBase.scala 中体现音乐数据的层次结构？

A1:

- **回答:**

数据模型采用分层结构：最底层是 Note（单个音符），中间层是 Chord（由一组 Note 构成的和

弦，同时包含时间标记），最上层是 Measure（由多个 Chord 构成的小节）。这种层次化设计体现了音乐的结构化组织。

Français :

Q1 : Comment la structure hiérarchique des données musicales est-elle représentée dans DataBase.scala ?

R1 :

- **Réponse :**

La hiérarchie se présente de la façon suivante : la couche la plus basse est constituée de Note (note individuelle), la couche intermédiaire de Chord (accord composé d'une liste de notes avec une indication temporelle) et la couche supérieure de Measure (mesure regroupant plusieurs accords). Cette conception hiérarchique reflète l'organisation structurée de la musique.

问题 2

中文:

Q2: DataBaseActor 中定义的 GetMeasure 消息参数的意义是什么？

A2:

- **回答:**

GetMeasure 消息的参数是一个整数索引，用于指明请求哪一个小节的数据。通过该索引，数据库可以从存储的 measures1 列表中定位并返回相应的 Measure 对象。

Français :

Q2 : Quelle est la signification du paramètre passé dans le message GetMeasure dans DataBaseActor ?

R2 :

- **Réponse :**

Le paramètre du message GetMeasure est un entier qui indique l'indice de la mesure à récupérer. Cet indice permet à la base de données de localiser dans la liste measures1 la mesure correspondante à retourner.

问题 3

中文:

Q3: measures1 列表中存储的两个 Measure 实例说明了什么？

A3:

- **回答:**

measures1 列表提供了实际的音乐数据示例，每个 Measure 包含多个和弦，每个和弦中有不同的音符。这展示了如何在程序中构造和组织复杂的音乐数据结构，以供后续演奏或处理。

Français :

Q3 : Que démontrent les deux instances de Measure stockées dans la liste measures1 ?

R3 :

- **Réponse :**

La liste `measures1` fournit des exemples concrets de données musicales. Chaque instance de `Measure` contient plusieurs accords, et chaque accord est constitué de plusieurs notes. Cela montre comment construire et organiser des structures de données musicales complexes destinées à la reproduction ou au traitement.

3. Maestro.scala

问题 1

中文:

Q1: Maestro 如何利用两个随机数生成器来决定下一个小节的选择?

A1:

- **回答:**

Maestro 分别使用 `diceA` 和 `diceB` 生成随机数, 并通过计算 `diceA.nextInt(5) + diceB.nextInt(5)` 得到一个随机值, 该值代表了骰子组合的结果。此结果被用作参数传递给 `ArrangerActor`, 从而参与小节选择。

Français :

Q1 : Comment le Maestro utilise-t-il les deux générateurs aléatoires pour déterminer le choix de la prochaine mesure ?

R1 :

- **Réponse :**

Le Maestro utilise deux générateurs aléatoires, `diceA` et `diceB`, et calcule l'indice en faisant `diceA.nextInt(5) + diceB.nextInt(5)`. Ce résultat, simulant le lancer de deux dés, est ensuite utilisé comme paramètre envoyé à l'acteur `ArrangerActor` pour la sélection de la mesure.

问题 2

中文:

Q2: Maestro 如何利用调度器 (scheduler) 实现音乐演奏的连续性?

A2:

- **回答:**

当 Maestro 接收到 `Measure` 消息后, 会将其转发给 `PlayerActor` 播放, 同时使用 `scheduler.scheduleOnce` 在预设的时间间隔 (`baseTime`) 后再次向自己发送 `StartPerformance` 消息, 从而触发下一次小节请求, 实现循环演奏。

Français :

Q2 : Comment le Maestro utilise-t-il le scheduler pour assurer la continuité de la performance musicale ?

R2 :

- **Réponse :**

Une fois que le Maestro reçoit un message Measure et le transmet au PlayerActor, il utilise scheduler.scheduleOnce pour s'auto-programmer l'envoi d'un message StartPerformance après un délai défini (baseTime). Cela permet de déclencher la demande de la prochaine mesure de manière cyclique, assurant ainsi la continuité de la performance.

问题 3

中文:

Q3: 为什么 Maestro 需要分别创建 PlayerActor 和 ArrangerActor 这两个子演员?

A3:

- **回答:**

这种设计将不同的职责模块化: ArrangerActor 专注于根据骰子值和预设矩阵选择合适的小节, 而 PlayerActor 专注于实际播放音符。分工明确有助于代码的可维护性和扩展性。

Français :

Q3 : Pourquoi le Maestro crée-t-il deux acteurs distincts, PlayerActor et ArrangerActor ?

R3 :

- **Réponse :**

La séparation des responsabilités est la raison principale : ArrangerActor se charge de la sélection de la mesure à partir des matrices et du calcul basé sur le dé, tandis que PlayerActor est responsable de la reproduction musicale en envoyant les messages MIDI. Cette modularité facilite la maintenance et l'extension du code.

4. Main.scala

问题 1

中文:

Q1: 在 Main.scala 中, Concert 对象的作用是什么?

A1:

- **回答:**

Concert 对象是程序的入口点。它负责初始化 ActorSystem、创建 Maestro 演员, 并通过发送 StartPerformance 消息启动整个音乐演奏流程。

Français :

Q1 : Quel est le rôle de l'objet Concert dans Main.scala ?

R1 :

- **Réponse :**

L'objet Concert constitue le point d'entrée de l'application. Il initialise l'ActorSystem, crée l'acteur Maestro, et lance la performance musicale en envoyant le message StartPerformance.

问题 2

中文：

Q2: ActorSystem 在 Main.scala 中是如何被创建的？系统名称是什么？

A2:

- **回答：**
ActorSystem 是通过调用 ActorSystem("MozartDiceGameSystem") 创建的，其名称为 "MozartDiceGameSystem"，这标识了当前的 Akka 系统实例。

Français :

Q2 : Comment l'ActorSystem est-il créé dans Main.scala et quel est son nom ?

R2 :

- **Réponse :**
L'ActorSystem est créé en appelant ActorSystem("MozartDiceGameSystem"), et son nom est "MozartDiceGameSystem", identifiant ainsi l'instance du système Akka.

问题 3

中文：

Q3: Main.scala 启动流程中向哪个演员发送了消息来开始表演？

A3:

- **回答：**
Main.scala 向 Maestro 演员发送 StartPerformance 消息，Maestro 接收到该消息后便开始调度和管理整个音乐表演流程。

Français :

Q3 : À quel acteur Main.scala envoie-t-il un message pour lancer la performance ?

R3 :

- **Réponse :**
Main.scala envoie le message StartPerformance à l'acteur Maestro, qui à son tour déclenche la chaîne d'événements pour démarrer la performance musicale.

5. Player.scala

问题 1

中文：

Q1: PlayerActor 如何获取并初始化 "Gervill" 合成器？

A1:

- **回答：**
PlayerActor 首先调用 MidiSystem.getMidiDeviceInfo() 遍历 MIDI 设备信息，并查找名称为 "Gervill" 的设备。如果找到，则使用 MidiSystem.getMidiDevice 获取该设备；若未找到，则打印

错误信息并退出程序。随后，通过调用 `device.open()` 打开设备，并通过 `device.getReceiver()` 获取用于发送 MIDI 消息的接收器。

Français :

Q1 : Comment le PlayerActor récupère-t-il et initialise-t-il le synthétiseur "Gervill" ?

R1 :

- **Réponse :**

Le PlayerActor utilise `MidiSystem.getMidiDeviceInfo()` pour parcourir les informations des appareils MIDI et recherche celui dont le nom est "Gervill". S'il est trouvé, il est récupéré via `MidiSystem.getMidiDevice`; sinon, un message d'erreur est affiché et le programme s'arrête. Ensuite, l'appareil est ouvert avec `device.open()` et son récepteur est obtenu avec `device.getReceiver()` pour permettre l'envoi de messages MIDI.

问题 2

中文:

Q2: PlayerActor 如何利用调度器 (scheduler) 安排 MIDI 消息的发送?

A2:

- **回答:**

当 PlayerActor 接收到 `MidiNote` 消息后，它使用 `scheduler.scheduleOnce` 方法：

- 根据音符的 `start` 属性安排在指定时间发送 `sendNoteOn` (启动音符)，
- 再根据 `start + dur` 安排发送 `sendNoteOff` (关闭音符)。

这种机制确保了 MIDI 消息在正确的时刻发送，实现了定时播放。

Français :

Q2 : Comment le PlayerActor utilise-t-il le scheduler pour programmer l'envoi des messages MIDI ?

R2 :

- **Réponse :**

Dès que le PlayerActor reçoit un message `MidiNote`, il utilise `scheduler.scheduleOnce` pour programmer :

- L'envoi de `sendNoteOn` à l'instant spécifié par `start` (pour activer la note),
- L'envoi de `sendNoteOff` à l'instant `start + dur` (pour désactiver la note).

Cela permet d'assurer que les messages MIDI sont envoyés au moment adéquat pour une lecture synchronisée.

问题 3

中文:

Q3: 在 PlayerActor 中，`sendNoteOn` 和 `sendNoteOff` 方法实现的关键细节是什么?

A3:

- **回答:**

这两个方法都通过创建 `ShortMessage` 对象并调用 `setMessage` 来设置 MIDI 消息参数 (包括消息类型、通道、音符和音量)。

- 对于 sendNoteOn，设置消息类型为 NOTE_ON 并传递有效的音量；
 - 对于 sendNoteOff，同样使用 NOTE_ON 但将音量设为 0，以模拟音符停止。
- 最后，通过调用 receiver.send 将消息发送到 MIDI 设备。

Français :

Q3 : Quels sont les points clés dans l'implémentation des méthodes sendNoteOn et sendNoteOff dans PlayerActor ?

R3 :

- **Réponse :**

Les deux méthodes créent un objet ShortMessage et utilisent setMessage pour configurer les paramètres du message MIDI (type de message, canal, note, vitesse, etc.).

- sendNoteOn configure le message avec le type NOTE_ON et une vitesse non nulle pour activer la note,
- Tandis que sendNoteOff utilise également NOTE_ON mais avec une vitesse de 0 pour simuler l'arrêt de la note.

Enfin, le message est envoyé via receiver.send au synthétiseur.

